



**UNIVERSIDAD ARGENTINA DE LA EMPRESA**  
Facultad de Ingeniería y Ciencias Exactas - FAIN

**Materia:** PROCESO DE DESARROLLO DE SOFTWARE

**Trabajo Práctico Obligatorio – Proceso de Desarrollo de Software**

**Fase Final: Implementación y prueba**

**20-06-2026 - Buenos Aires, Argentina**

**Integrantes del grupo:**

Cortés, Ignacio 1203485  
Etchart, Facundo 1201391  
Rubachin, Sofía 1196599

**Profesor:**

***Ing. Leon Maria Angela***

---

# Índice

|                                                        |          |
|--------------------------------------------------------|----------|
| <b>Índice.....</b>                                     | <b>1</b> |
| <b>Enlace al repositorio.....</b>                      | <b>2</b> |
| <b>1. Descripción general de la problemática.....</b>  | <b>2</b> |
| <b>2. Requerimientos funcionales resueltos.....</b>    | <b>2</b> |
| <b>3. Justificación del diseño arquitectónico.....</b> | <b>2</b> |
| Separación de responsabilidades.....                   | 2        |
| Patrones GoF aplicados.....                            | 3        |
| Facade.....                                            | 3        |
| Factory Method / Simple Factory.....                   | 3        |
| State.....                                             | 3        |
| Observer.....                                          | 4        |
| Principios SOLID aplicados.....                        | 4        |
| Patrones GRASP aplicados.....                          | 4        |
| <b>4. Instrucciones de despliegue.....</b>             | <b>4</b> |
| <b>5. Evidencia de ejecución.....</b>                  | <b>5</b> |
| <b>6. Conclusiones.....</b>                            | <b>6</b> |

# Sistema de Gestión Operativa para un Barrio Cerrado

## Enlace al repositorio

Repositorio del proyecto: <https://github.com/sofiruba/eBarrio>

## 1. Descripción general de la problemática

eBarrio es un sistema de gestión operativa para un barrio cerrado. El objetivo fue centralizar en una aplicación Java la administración de residentes, viviendas, visitantes, proveedores, accesos, reclamos, tareas de mantenimiento, incidentes de seguridad, usuarios y notificaciones.

El problema original era la dispersión de información entre administración, seguridad, mantenimiento y residentes. El sistema resuelve ese escenario mediante una interfaz JavaFX con login por rol, vistas separadas para administrador y residente, persistencia en archivos JSON y una fachada central (SistemaBarrio) que concentra los flujos de negocio.

## 2. Requerimientos funcionales resueltos

- Inicio de sesión con usuarios administradores y residentes.
- Vista de administrador con gestión de residentes, viviendas, solicitudes, accesos, visitantes, proveedores y notificaciones. Además de una sección para seguridad y mantenimiento.
- Vista de residente con dashboard propio, visitantes/proveedores autorizados y solicitudes propias.
- Alta y modificación de residentes, con creación automática de cuenta de usuario.
- Persistencia de usuarios, residentes, viviendas, visitantes, accesos, solicitudes y notificaciones en JSON.
- Registro, edición, eliminación y filtrado de visitantes y proveedores.
- Registro de ingreso y egreso con fecha y hora.
- Patente opcional para visitantes/proveedores.
- Validaciones de datos de entrada: documento, email, teléfono y patente.
- Creación de reclamos, tareas de mantenimiento e incidentes de seguridad.
- Avance y cancelación de solicitudes mediante estados.
- Generación de notificaciones ante ingresos, egresos y solicitudes.
- Escenarios de prueba por consola para demostrar flujos principales y alternativos.

## 3. Justificación del diseño arquitectónico

### Separación de responsabilidades

- app: contiene la interfaz JavaFX y las vistas principales.

- sistema: contiene SistemaBarrio, fachada/controlador de casos de uso.
- model: contiene entidades de dominio, reglas de negocio y patrones aplicados.
- data: contiene los archivos JSON usados como base de datos simple.
- documentación: contiene informe, UML, escenarios y registro de commits.

## **Patrones GoF aplicados**

### **Facade**

Problemática detectada: la interfaz necesitaba ejecutar muchas operaciones sobre residentes, visitantes, accesos, solicitudes, usuarios, notificaciones y persistencia. Si cada pantalla conocía todas las clases internas, el acoplamiento crecía demasiado.

Solucion con el patron: SistemaBarrio expone metodos como registrarResidente, registrarVisitante, registrarAcceso, crearReclamo, avanzarEstadoSolicitud, autenticarUsuario, cargarDesdeJson y métodos de guardado. La vista invoca la fachada y no manipula directamente la persistencia ni la creación compleja de objetos.

Ventaja obtenida: menor acoplamiento entre UI y dominio, punto único de entrada para los casos de uso y mayor facilidad para probar los flujos por consola.

### **Factory Method / Simple Factory**

Problemática detectada: las solicitudes comparten datos y comportamiento, pero existen tipos concretos distintos: reclamos, tareas de mantenimiento e incidentes de seguridad.

Solución con el patrón: SolicitudFactory.crearSolicitud(...) recibe el tipo y crea la subclase correspondiente: Reclamo, TareaMantenimiento o IncidenteSeguridad.

Ventaja obtenida: se centraliza la creación de solicitudes y se evita distribuir condicionales de construcción en varias partes de la aplicación.

### **State**

Problemática detectada: una solicitud cambia de estado siguiendo reglas específicas: Pendiente, Asignado, En proceso, Resuelto y Cerrado. Cada estado habilita o limita acciones distintas.

Solución con el patrón: Solicitud mantiene una referencia a IEstadoSolicitud. Las clases EstadoPendiente, EstadoAsignado, EstadoEnProceso, EstadoResuelto y EstadoCerrado implementan las transiciones de avanzar y cancelar.

Ventaja obtenida: las reglas de transicion quedan encapsuladas en clases pequeñas, cohesivas y fáciles de modificar sin llenar Solicitud de condicionales.

## Observer

Problemática detectada: cuando cambia una solicitud, otros actores del sistema pueden necesitar enterarse sin que la solicitud dependa de clases concretas.

Solución con el patrón: Solicitud mantiene una lista de IObservador y notifica con notificarObservadores. Administrador implementa IObservador.

Ventaja obtenida: bajo acoplamiento entre solicitud y observadores, con posibilidad de agregar nuevos observadores sin cambiar la clase Solicitud.

## Principios SOLID aplicados

- SRP: Acceso controla ingreso/egreso, Visitante representa autorizados, Notificación representa mensajes, los estados gestionan transiciones y SistemaBarrio coordina casos de uso.
- OCP: se pueden agregar nuevos tipos de solicitud extendiendo Solicitud y agregando su creación en SolicitudFactory.
- LSP: Reclamo, TareaMantenimiento e IncidenteSeguridad pueden usarse como Solicitud sin romper los flujos.
- ISP: IObservador expone sólo el método actualizar, suficiente para el patrón Observer.
- DIP: Solicitud depende de la abstracción IObservador, no de un observador concreto.

## Patrones GRASP aplicados

- Controller: SistemaBarrio recibe las acciones principales desde la interfaz y las pruebas.
- Creator: SistemaBarrio crea entidades del flujo y SolicitudFactory crea solicitudes concretas.
- Information Expert: Acceso sabe si está activo, Residente administra sus visitantes, Solicitud delega su avance al estado actual.
- Low Coupling: la vista trabaja contra la fachada y las solicitudes observan IObservador.
- High Cohesion: los paquetes separan barrio, accesos, solicitudes, notificaciones, vistas y datos.

Vinculación con SOLID: Controller y Low Coupling se relacionan con DIP; High Cohesion se relaciona con SRP; Creator se apoya en SRP y OCP; Information Expert reduce acoplamiento al ubicar comportamiento cerca de los datos que lo necesitan.

## 4. Instrucciones de despliegue

Requisitos:

- JDK instalado.
- JavaFX SDK 17 disponible. El repositorio incluye eBarrio/libs/javafx-sdk-17.0.19/lib.

Compilar desde la raíz del repositorio:

```
javac --module-path eBarrio/libs/javafx-sdk-17.0.19/lib --add-modules javafx.controls  
-encoding UTF-8 -d out $(find eBarrio/src -name '*.java')
```

Ejecutar interfaz JavaFX:

```
java --module-path eBarrio/libs/javafx-sdk-17.0.19/lib --add-modules javafx.controls -cp out  
app.MainApp
```

Ejecutar pruebas por consola:

```
java -cp out Main  
java -cp out model.solicitud.test.TestReclamo  
java -cp out model.solicitud.test.EscenariosEjecucion
```

Usuarios de prueba:

- Administrador: admin@ebarrio.com / admin123
- Residente: sofia@email.com / sofia123
- Residente: martin@email.com / martin123
- Residente: ana@email.com / ana123
- Residente: facu@gmail.com / 34567891

## 5. Evidencia de ejecución

Compilación verificada el 20/06/2026:

```
javac --module-path eBarrio/libs/javafx-sdk-17.0.19/lib --add-modules javafx.controls  
-Xlint:unchecked -encoding UTF-8 -d /tmp/ebarrio-docs $(find eBarrio/src -name '*.java')
```

Resultado: sin errores de compilación.

Ejecución de TestReclamo:

*Estado de solicitud [1]: Pendiente*  
*La solicitud pasa de 'Pendiente' a 'Asignado'.*  
*La solicitud pasa de 'Asignado' a 'En proceso'.*  
*La solicitud pasa de 'En proceso' a 'Resuelto'.*  
*La solicitud pasa de 'Resuelto' a 'Cerrado'.*  
*El estado de la solicitud no puede avanzar: ya fue cerrado.*  
*La solicitud que se encontraba pendiente de asignación ha sido directamente cerrada.*  
*La solicitud ya está cerrada y no se puede cancelar.*

Ejecución de EscenariosEjecucion:

*Ingreso registrado para Camila Ruiz*  
*Egreso registrado para Camila Ruiz*  
*Reclamo creado: Reclamo [1] Luminaria rota*

La solicitud pasa de 'Pendiente' a 'Asignado'.  
 La solicitud pasa de 'Asignado' a 'En proceso'.  
 La solicitud pasa de 'En proceso' a 'Resuelto'.  
 La solicitud pasa de 'Resuelto' a 'Cerrado'.  
 Tarea de mantenimiento creada: TareaMantenimiento [2] Cortar pasto  
 La solicitud que se encontraba pendiente de asignación ha sido directamente cerrada.  
 Todos los escenarios de ejecución finalizaron correctamente.

Capturas de la interfaz gráfica:

Login:

Vista residente:

**eBarrio**

**eBarrio**

Panel residente

Mi inicio

Mis visitantes

Mis reclamos

Nuevo reclamo

Notificaciones: 0

Sofia Gomez

Cerrar sesion

### Mis visitantes

Personas autorizadas para ingresar a tu vivienda

Ver: Todos

Autorizar visitante/proveedor

Editar

Eliminar

#### Visitantes autorizados

| Tipo      | Nombre      | DNI      | Patente | Frecuencia | Motivo          |
|-----------|-------------|----------|---------|------------|-----------------|
| Visitante | Camila Ruiz | 42123123 | AB123CD | Unica vez  | Visita familiar |
|           |             |          |         |            |                 |
|           |             |          |         |            |                 |
|           |             |          |         |            |                 |
|           |             |          |         |            |                 |
|           |             |          |         |            |                 |
|           |             |          |         |            |                 |

Vista administrador:

**eBarrio**

**eBarrio**

Panel administrador

Inicio

Residentes

Accesos

Solicitudes

Seguridad

Mantenimiento

Notificaciones: 4

Administrador

Cerrar sesion

### Bienvenido, Administrador

Resumen general del barrio

1  
Residentes

1  
Viviendas

1  
Visitantes

2  
Solicitudes

#### Solicitudes recientes

Ver todas

| ID | Descripcion    | Solicitante   | Prioridad | Estado  | Fecha      |
|----|----------------|---------------|-----------|---------|------------|
| #2 | Cortar pasto   | Mantenimiento | Media     | Cerrado | 2026-06-20 |
| #1 | Luminaria rota | Sofia Gomez   | Alta      | Cerrado | 2026-06-20 |
|    |                |               |           |         |            |
|    |                |               |           |         |            |
|    |                |               |           |         |            |
|    |                |               |           |         |            |

**eBarrio**

Notificaciones: 4

Administrador

Cerrar sesion

**eBarrio**

Panel administrador

Inicio

Residentes

Accesos

Solicitudes

Seguridad

Mantenimiento

## Residentes

Alta y consulta de residentes del barrio

Nuevo residente

Editar residente

Listado de residentes

| Nombre      | DNI      | Vivienda | Telefono   |
|-------------|----------|----------|------------|
| Sofia Gomez | 40111222 | Lote 12  | 1130000000 |
|             |          |          |            |
|             |          |          |            |
|             |          |          |            |
|             |          |          |            |
|             |          |          |            |
|             |          |          |            |

**eBarrio**

Notificaciones: 4

Administrador

Cerrar sesion

**eBarrio**

Panel administrador

Inicio

Residentes

Accesos

Solicitudes

Seguridad

Mantenimiento

## Accesos y visitantes

Visitantes autorizados, ingresos y egresos del barrio

Ver 

Todos

Nuevo visitante/proveedor

Editar

Eliminar

Visitantes autorizados

| Tipo      | Nombre      | DNI      | Patente | Frecuencia | Motivo          |
|-----------|-------------|----------|---------|------------|-----------------|
| Visitante | Camila Ruiz | 42123123 | AB123CD | Unica vez  | Visita familiar |
|           |             |          |         |            |                 |
|           |             |          |         |            |                 |
|           |             |          |         |            |                 |
|           |             |          |         |            |                 |
|           |             |          |         |            |                 |

**eBarrio**

Notificaciones: 4

Administrador

Cerrar sesion

**eBarrio**

Panel administrador

Inicio

Residentes

Accesos

Solicitudes

Seguridad

Mantenimiento

## Solicitudes

Reclamos, tareas e incidentes

Nuevo reclamo

Nueva tarea

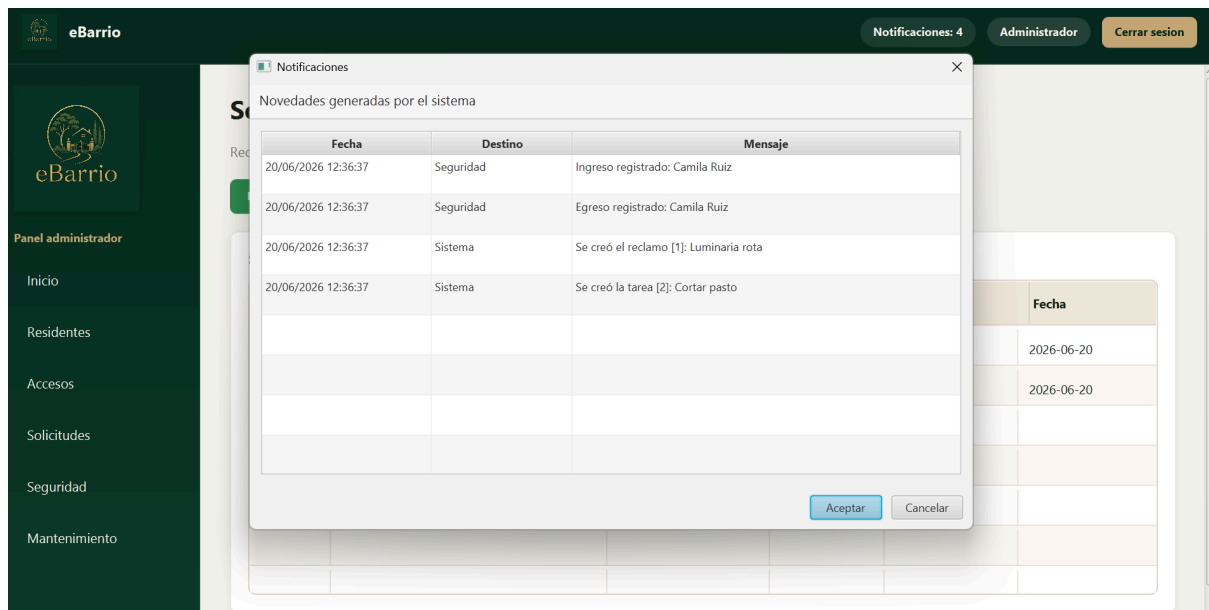
Nuevo incidente

Avanzar estado

Cancelar

Seguimiento de solicitudes

| ID | Descripcion    | Solicitante   | Prioridad | Estado  | Fecha      |
|----|----------------|---------------|-----------|---------|------------|
| #1 | Luminaria rota | Sofia Gomez   | Alta      | Cerrado | 2026-06-20 |
| #2 | Cortar pasto   | Mantenimiento | Media     | Cerrado | 2026-06-20 |
|    |                |               |           |         |            |
|    |                |               |           |         |            |
|    |                |               |           |         |            |
|    |                |               |           |         |            |



## 6. Reflexión breve por integrante

### Etchart, Facundo

Armar el módulo de solicitudes me sirvió un montón para ver el patrón State en acción. Al principio parecía medio rebuscado, pero cuando vi que la jerarquía de Reclamos y Tareas podía cambiar de estado sin llenarse de sentencias *if-e/se* por todos lados, entendí en la práctica por qué se usa tanto. Además, hacer el ejercicio de justificar estas decisiones cruzándolas con los principios SOLID y GRASP ayudó bastante a bajar toda la teoría de la materia a un problema real, logrando un código mucho más limpio y fácil de mantener. Además usar el github y llevar a cabo los commits y documentación me hizo mantener un buen control de las tareas y de lo que estamos haciendo

### Cortés, Ignacio

Me tocó agarrar la clase SistemaBarrio y aplicarle el patrón Facade, lo cual fue un buen desafío porque tuve que centralizar casi toda la lógica del negocio para que la interfaz gráfica la pueda consumir sin acoplarse de más. Sumado a eso, aplicar el patrón Observer para automatizar las notificaciones y gestionar los accesos me dejó clarísimo que separar responsabilidades no es solo para cumplir con la teoría, sino que literalmente te salva de que el código se vuelva inmanejable a medida que el proyecto va creciendo. También usar JavaFX y ver como quedan las cosas es muy satisfactorio y así agregar dashboards y tablas.

### Rubachin, Sofía

Hacer la estructura inicial del dominio con Residentes, Viviendas y Visitantes me dio una visión re clara de cómo iba a circular la información en el sistema. Pero lo que más rescato fue llevar ese modelo a la interfaz visual con JavaFX; lograr que los datos finalmente se muestren bien en las tablas fue una gran satisfacción. Por otro lado, llevar el control de los commits en Git y ordenar la documentación me hizo dar cuenta de que, por más buen

código que hagas, si el equipo no está coordinado y el repositorio no está prolijo, el trabajo final no se luce.

## 7. Conclusiones

El proyecto permitió integrar conceptos de orientación a objetos, patrones de diseño, persistencia simple e interfaz gráfica en un dominio concreto. La mayor dificultad fue mantener coherencia entre el modelo, la interfaz y los datos persistidos, especialmente cuando se agregaron roles, usuarios, visitantes/proveedores y accesos.

Los patrones aportaron orden al diseño: Facade simplifico el uso del sistema desde la interfaz, State evitó condicionales extensos para las solicitudes, Factory concentró la creación de tipos de solicitud y Observer permite desacoplar notificaciones de cambios. Como desventaja, aplicar patrones aumenta la cantidad de clases y exige mantener el UML actualizado, pero el resultado final es más claro, extensible y defendible.

Como equipo, se concluye que el sistema alcanza los objetivos funcionales principales de la problemática y deja una base ordenada para futuras mejoras, como usar una base de datos real, agregar autenticación más robusta y exportar reportes de administración.